

ALICE manual

Version 1.1

Jeffrey Durieux, Elise Dusseldorp, Wouter van Loon & Juan Claramunt

03/07/2025

1 Introduction

In this document, we describe the set-up of a simulation study and the performance of it on a cluster computer (ALICE). Also, we introduce basic knowledge on how to use the ALICE cluster.

An example is included to illustrate each step of the simulation study and its computation using a cluster computer.

The document is structured as follows:

- Chapter 2: How to connect to the ALICE cluster;
- Chapter 3: File management on a cluster;
- Chapter 4: Specification of a simulation study;
- Chapter 5: Preparation of a simulation study;
- Chapter 6: Performance of a simulation study on a cluster computer;
- Chapter 7: Collecting the results from the cluster computer in one large matrix and evaluation of the results by ANOVA;
- Chapter 8: Email notifications;
- Chapter 9: Installing R packages on ALICE;
- Chapter 10: Warning for writing shell scripts on Windows;
- Chapter 11: Useful Linux commands.
- Chapter 12: Matlab

Note: *You may find all functions required for the example in the MS-Github page [click here](#)*

2 How to connect to the ALICE cluster

There are several options to connect to ALICE. They depend on your operating system (Windows, Linux, Mac OS, etc.).

Details of the connection:

Research Jump server (Proxy server):

Hostname: ssh-gw.alice.universiteitleiden.nl

Port: 22

2.0.1 Windows users

Connect with Putty

1. Open putty.
2. Introduce hostname (see details above).

3. Select ssh.
4. (Optional) Set a name under saved sessions and save.
5. (Optional) At the connection setting, fill in 60 at the Seconds between keepalives. If needed, enable the Enable TCP keepalives.
6. (Optional) At the X11 setting, you can enable the Enable X11 forwarding. Do this if you need graphical output, but for this to get this working, you need to install separately a X11 server for Windows.
7. Left click on open.
8. Introduce your ALICEusername.
9. Introduce your password.
10. Type: `ssh ALICEusername@login1.alice.universiteitleid.nl`
11. Introduce your password again.
12. Congratulations! You are now in ALICE.

Connect with MobaXterm (recommended)

First option (the session is saved for future connections):

1. Open MobaXterm.
2. Click on sessions.
3. Click on ssh.
4. Type 'login1.alice.universiteitleid.nl' in 'Remote host'.
5. Select specify username and introduce your ALICEusername, and set port to 22.
6. (Optional) Check the 'Follow SSH path' in the 'Advanced SSH settings'-TAB.
7. Check the 'Connect through SSH gateway (jump host)' in the 'Network settings'-TAB.
8. Fill the gateway SSH server (`ssh-gw.alice.universiteitleid.nl`), the port (22) and the user (ALICEusername) in the 'Network settings'-TAB.
9. (Optional) Un-check the 'Display reconnection message at session end' in the 'Bookmark settings'-TAB.
10. Click on OK.
11. Introduce your password.
12. Once in the terminal, introduce your password again.
13. Congratulations! You are now in ALICE.

Second option (faster)

1. Open MobaXterm
2. Click on start local terminal.
3. Type: `ssh ALICEusername@ssh-gw.alice.universiteitleid.nl`
4. Introduce your password.
5. Type: `ssh ALICEusername@login1.alice.universiteitleid.nl`
6. Introduce your password again.
7. Congratulations! You are now in ALICE.

2.0.2 Linux users

Recommended option by the maintainers of ALICE:

Before connecting the first time follow the next steps:

1. Open a Terminal and create a directory (if needed) `.ssh` in your home directory.
2. Create a config file inside your `.ssh` directory and add the following: `vi ~/.ssh/config`
3. Add also:


```
Host login1.alice.universiteitleid.nl
    Hostname login1.alice.universiteitleid.nl
    Localforward 6001 login1.alice.universiteitleid.nl:22
    User <ALICEusername>
    ProxyCommand ssh -q -X -C ALICEusername@ssh-gw.alice.universiteitleid.nl nc %h %p
```

4. Set the permissions for the user for the .ssh directory: `chmod 700 ~/.ssh`
5. Set the permissions for the user on all the files in your .ssh directory: `chmod 600 ~/.ssh/*`

Once the previous steps are finished, follow the next ones to connect:

1. Type `ssh login1.alice.universiteitleid.nl`
2. Introduce your password.
3. Introduce your password again.
4. Congratulations! You are now in ALICE.

Faster option (does not require the config file):

1. Open a Terminal.
2. Type `ssh -X ALICEusername@ssh-gw.alice.universiteitleid.nl`
3. Type `ssh login1.alice.universiteitleid.nl`.
4. Congratulations! You are now in ALICE.

You can also install X2Go (similar to mobaXterm).

2.0.3 Mac users

Recommended option by the maintainers of ALICE.

Before connecting the first time follow the following steps:

1. Open a Terminal and create a directory (if needed) .ssh in your home directory.
2. Create a config file inside your .ssh directory and add the following: `vi ~/.ssh/config`
3. Add also:


```
Host login1.alice.universiteitleid.nl
    Hostname login1.alice.universiteitleid.nl
    Localforward 6001 login1.alice.universiteitleid.nl:22
    User <ALICEUsername>
    ProxyCommand ssh -q -X -C ALICEusername@ssh-gw.alice.universiteitleid.nl nc %h %p
```
4. Set the permissions for the user for the .ssh directory: `chmod 700 ~/.ssh`
5. Set the permissions for the user on all the files in your .ssh directory: `chmod 600 ~/.ssh/*`

Once the previous steps are finished, follow the next ones to connect:

1. Type `ssh login1.alice.universiteitleid.nl`
2. Introduce your password.
3. Introduce your password again.
4. Congratulations! You are now in ALICE.

Fast option (does not require the config file):

1. Open a Terminal.
2. Type `ssh -Y ALICEusername@ssh-gw.alice.universiteitleid.nl`
3. Type `ssh login1.alice.universiteitleid.nl`.
4. Congratulations! You are now in ALICE.

You can find more information [here](#).

3 File management in a cluster

The file management (copy, move, upload, download files/folders) also depends on your operating system.

3.1 Windows users

Windows users need a SSH File Transfer Protocol (SFTP) client software. MobaXterm (recommended SSH client software) includes a SFTP, however, Putty does not. Therefore, in case you use Putty, other software

such as Filezilla or WinSCP is needed.

3.1.1 MobaXterm users

The SFTP in **MobaXterm** can be found in the TABs on the left side (see Figure 1). In order to use it, you must connect to ALICE first. Once in ALICE, the SFTP tab contains icons to:

- Move to the parent directory
- Download files
- Upload files
- Refresh the folder
- Create a new directory
- Create a new file
- Delete a new file

Below these icons there is the path corresponding to your current folder (directory) in ALICE.

Below the path you can find the files and folders in the current path.

3.1.2 Putty users

If you are using **Putty** to connect to ALICE, the process is far more difficult. It is necessary to download a software such as WinSCP and create an SSH tunnel.

First **WinSCP** is installed. Click on New Session. Go to advanced settings and select tunnel on the left side (see Figure 3). Select “connect through SSH tunnel”. Fill the hostname (ssh-gw.alice.universiteitleiden.nl), Port number (22), user name (ALICEusername) and password (ALICEpassword) and click on OK. Then, fill in the new window the hostname (login1.alice.universiteitleiden.nl), Port number (22), user name (ALICEusername) and password (ALICEpassword).

NOTE: It is important to realize that the first hostname is not the same as the second host name!

3.2 Linux and Mac users

Open the terminal and use the command **scp** to transfer files to/from the cluster.

The basic syntax of scp is:

scp [from] [to]

where “from” can be a filename or a folder and “to” contains your netid, the hostname of the cluster login node, and the destination directory if you are tranfering files to ALICE and vice versa if you are downloading files from ALICE. For example, to upload files:

scp myfile.txt ALICEusername@login1.alice.universiteitleiden.nl:/exports/fsw

In this example, “myfile.txt” is copied to the directory /exports/fsw of ALICE.

If you want to transfer the whole folder with its content you need to add -r, for example:

scp -r mydirectory ALICEusername@login1.alice.universiteitleiden.nl:/exports/fsw

To download files/folder from ALICE just interchange the paths:

scp ALICEusername@login1.alice.universiteitleiden.nl:/exports/fsw/myfile.txt MyDocuments

In this example, myfile.txt is downloaded from ALICE to your local folder MyDocuments.

Remark: In ALICE, the files must be transfered through the jump host. This means that you need to transfer the files from your PC/laptop to the jump host using scp and then from the jump host to ALICE (or vice versa if you want to download files from ALICE). Let’s see how to do it with two examples:

Transfer the file text.txt from your laptop/PC to ALICE.

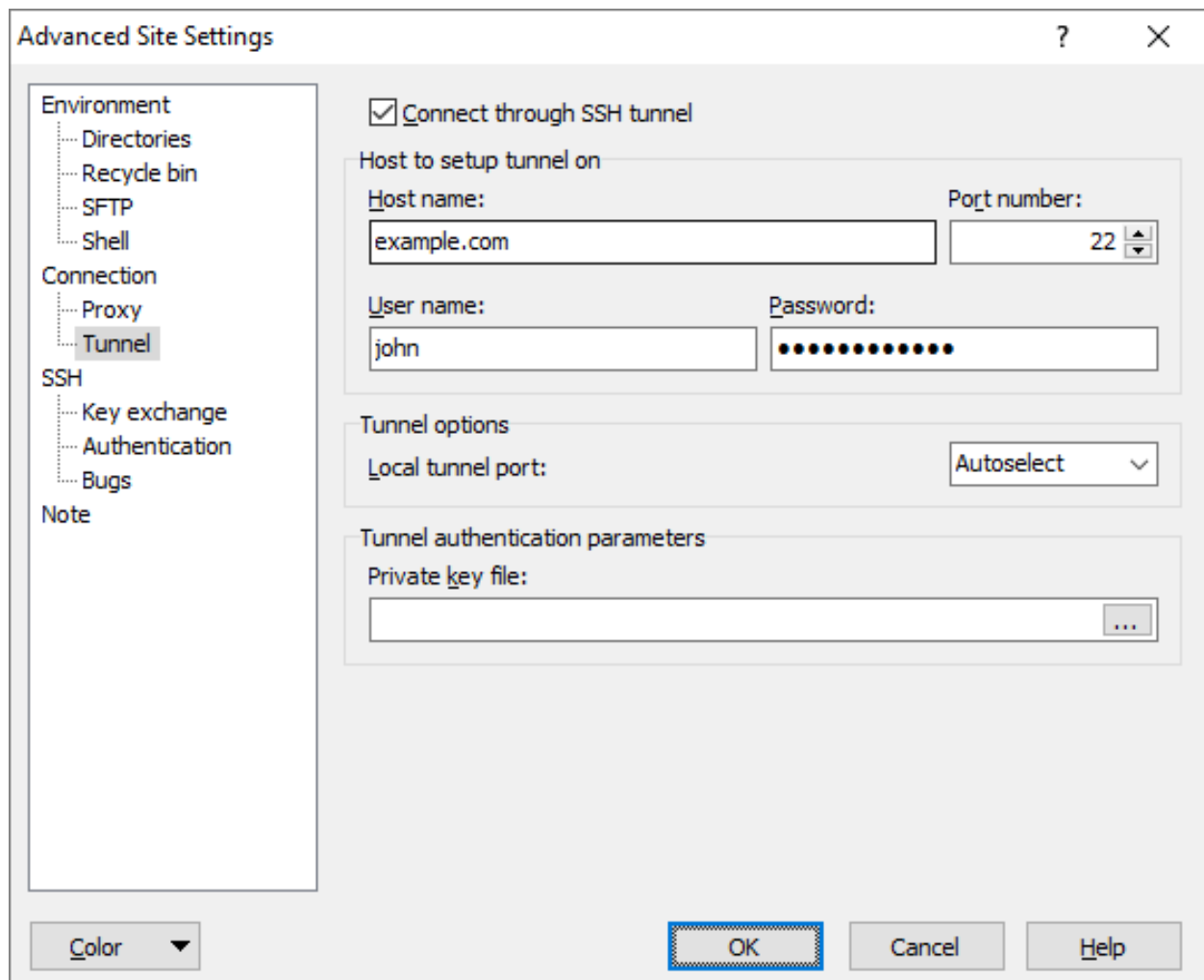


Figure 2: WinSCP advance options window

First, transfer the file to the jump host:

```
scp text.txt ALICEusername@ssh-gw.alice.universiteitleid.nl:/home/ALICEusername
```

Second, connect to the jump host typing

```
ssh ALICEusername@ssh-gw.alice.universiteitleid.nl
```

Third, transfer the file to ALICE (home folder):

```
scp text.txt ALICEusername@login1.alice.universiteitleid.nl:/home/ALICEusername
```

Optional: Remove the files from the jump host.

Transfer the file text.txt from ALICE to your laptop/PC.

First, connect to the jump host:

```
ssh ALICEusername@ssh-gw.alice.universiteitleid.nl
```

Second, transfer the file from ALICE (home directory) to the jump host:

```
scp ALICEusername@login1.alice.universiteitleid.nl:/home/ALICEusername/text.txt  
/home/ALICEusername
```

Third, exit the jump host. Go back to your own PC/laptop terminal.

Fourth, transfer the file from the jump host to your laptop by typing on your terminal:

```
scp ALICEusername@ssh-gw.alice.universiteitleid.nl:/home/ALICEusername/text.txt  
MyDocuments
```

Optional: Remove the files from the jump host.

Alternatively, if you have created the **login1.alice.universiteitleid.nl** config file to connect it is possible to transfer the files without using the scp command twice:

Transfer the file text.txt from laptop/PC to your ALICE.

Open a terminal and type:

```
scp text.txt login1.alice.universiteitleid.nl:/home/ALICEusername
```

Transfer the file text.txt from ALICE to your laptop/PC.

Open a terminal and type:

```
scp login1.alice.universiteitleid.nl:/home/ALICEusername/text.txt MyDocuments/text.txt
```

3.2.1 Transfer the files using SFTP client for Mac (Fugu) and Linux (Filezilla, X2Go)

You can also transfer files between your local computer and a cluster using an SFTP client software, such as Fugu (OSX) or FileZilla (Linux).

For more information on how to transfer files from or to a cluster, *click here*.

Fugu (Mac users)

Start by install Fugu. Then, open Fugu and in the menu go to SSH and afterward New SSH tunnel (cmd T). Fill Create Tunnel to (login1.alice.universiteitleid.nl), Service or Port (22), Tunnel Host(ssh-gw.alice.universiteitleid.nl), Username (ALICEusername) and Port (22).

Next, type your password.

An additional window appears that displays your tunnel. The Local port was set automatically. Check this number (the local port), you will need it.

In the main window, fill out the field Connect to: with “localhost”, type your ALICEusername, and the number of your local port in the field “Port:”. Then, press connect, and enter your password.

On the left side of the screen you will have the files/folders on your computer and on the right side of the screen the files/folders on the ALICE cluster. You can now transfer files by dragging them between the left and right screen.

X2Go (Linux users)

While installing X2Go we can configure the protocol to transfer files. For more information on how to configure it, [click here](#).

3.3 Change your password

To change the password, you should type in the command line *passwd*. Then, enter your old password once and your new password twice.

Note: Remember to update your password in all the ALICE related software, otherwise you will probably get in the screen a warning message: *REMOTE HOST IDENTIFICATION HAS CHANGED!*. This warning also appears if the remote machine has a new update.

To solve this problem open the file where your key’s are saved and remove the key that is old. In the warning message it says which is the key to be changed. This means we need to remove the corresponding line from the file “known_hosts”.

4 Specification of a simulation study

In a full factorial simulation study, we consider the following 4 steps:

- Step 1: Research Question
- Step 2: Gauge and Design
- Step 3: Statistical Analysis
- Step 4: Evaluation Criteria

4.1 Step 1: Research Question

First, you formulate the research question you would like to answer by means of a simulation study.

In our example, the research questions are: Does the Mann-Whitney U test show higher power (i.e., lower Type II error) than the independent samples t-test? And does the difference in power between the two methods depend on the total sample size and the mean difference between the two groups?

We hypothesize that the power of the Mann-Whitney test is higher than the independent samples t-test, especially in smaller samples.

4.2 Step 2: Gauge and design

Start with the formulation of:

- The true scenario(s)/model(s) from which you will generate your artificial data, and specify from which distributions you generate the variables that are included in the model.
 - In our example, we have a continuous, standardized outcome variable and one variable group that has two categories (i.e., two groups).
- Specify the design factors (i.e. the factors you are going to vary systematically).

In our example, there are two design factors:

- Samp (sample size, having 4 levels: $N = 10, 20, 40$, and 80 in the example);
- Es (effect size, having 3 levels: $d = 0.2, 0.5$ and 0.8 in the example). In this example, the effect size is the standardized difference in means between the two groups.
- Number of replications within each cell: k . In our small example, $k = 10$.

Then, our design contains in total 4 (Samp) by 3 (Es) 12 cells, and for each cell we will generate $k = 10$ data sets. Thus, in total, we analyze 120 data sets.

4.3 Step 3: Statistical Analysis

Specify the type of analysis you want to perform for each dataset (of each cell of the design). Often you want to compare two types of analysis methods (a new method compared to an old method).

The two methods used in the statistical analysis of the example are the t-test and the Mann-Whitney test which can be found in the *Method_old.R* and *Method_new.R* files respectively.

4.4 Step 4: Evaluation Criteria

Specify the evaluation criteria which you want to use to compare the two methods.

In our example, the evaluation criterion used to compare both methods (the t-test and the Mann-Whitney) is the number of times that the test is significant using $\alpha = 0.05$. If it is significant it means that the test correctly rejects the null hypothesis of no difference in means between the two groups.

5 Preparation of a simulation study

Write the following functions in R:

- A function that generates one data set for each true scenario in the design. The input arguments of this function are the type of true model and the levels of the factors of the simulation design. In our example, we refer to this function as *MyDataGeneration.R*. The output of this function is a simulated data set (SimData);
- One or two functions that analyse the data according to one or two methods. In our example, we refer to these functions as *Method_old.R* and *Method_new.R*. The input argument of these functions is a dataset (and possibly more input arguments). The output of this function contains the relevant results. In our example, we save the output of these functions as MyAnalysisResult (see *MainSimulationScriptSLURM.R* below);
- A function that applies the evaluation criteria. In our example, we refer to this function as *MyEvaluation.R*. The input of this function is MyAnalysisResult, this is, the output of the methods.

Note: If you are a Matlab user, go to chapter 12.

6 Performance of a simulation study on a cluster computer

6.1 The main simulation script:

Below you can see the main simulation script called *MainSimulationScriptSLURM.R*. This is the script that contains your whole simulation study. Here, your design is coded, analyses are specified and evaluation criteria are computed. Moreover, saving your output to your directory is specified.

Some usefull and sometimes necessary functions:

- `commandArgs()`
 - captures supplied command line arguments (see section 6.2 and 6.3)
- `expand.grid()`

- usefull function that creates a data frame from all combinations of factor variables
- `set.seed()`
 - important for replication of your simulation study.
- `do.call()`
 - neat function to execute a function with a list of supplied arguments

```

1  #!/bin/bash
2  # MainSimulationScriptSLURM.R
3
4  # args is a stringvector
5  args <- commandArgs(TRUE)
6  args <- as.numeric(args)
7
8  RowOfDesign <- args[1]
9  Replication <- args[2]
10
11 ### Simulation design example
12 samp <- c(10, 20, 40, 80)
13 es <- c(0.2, 0.5, 0.8)
14
15 # Design is a data.frame with all possible combinations of the factor levels
16 # Each row of the design matrix represents a cell of your simulation design
17
18 Design <- expand.grid(samp = samp, es = es)
19
20 # set a random number seed to be able to replicate the result exactly
21 set.seed((Replication + 1000)*RowOfDesign)
22
23 ###Preparation:
24 ##Install R packages first in separate job file (see chapter 9)
25 #Always use library() herento activate the non-standard R-packages, for example
26 #library(ica)
27 #By the way: this is just to illustrate, we do not use this package for our example
28
29 source("/home/claramuntj1/ALICEtutorial/MyDataGeneration.R")
30 source("/home/claramuntj1/ALICEtutorial/Method_new.R")
31 source("/home/claramuntj1/ALICEtutorial/Method_old.R")
32 source("/home/claramuntj1/ALICEtutorial/MyEvaluation.R")
33
34 #An alternative to the previous lines is to create your own directory
35 #and upload the files to it. For example:
36 #source("/home/ALICEusername/yourdirectory/MyDataGeneration.R")
37 #source("/home/ALICEusername/yourdirectory/Method_new.R")
38 #source("/home/ALICEusername/yourdirectory/Method_old.R")
39 #source("/home/ALICEusername/yourdirectory/MyEvaluation.R")
40
41 #IMPORTANT WARNING!!!
42 #When creating your own simulation study,
43 #make sure to create your own directory on your Home directory.
44 #For example: /home/ALICEusername/projectName
45
46 ##### simulation #####
47 # Generate data
48 SimData <- do.call(MyDataGeneration, Design[RowOfDesign, ] )

```

```

49
50 # Analyze data set with Method_new
51 tmp <- proc.time()
52 MyAnalysisResult1 <- Method_new(SimData)
53
54 #Analyze data set with Method_old
55 MyAnalysisResult2 <- Method_old(SimData)
56 time <- proc.time() - tmp #save the time to run the analyses of one cell of the design.
57 # Also possible to time both analyses seperately
58
59 #Combine relevant results of the analysis by the two methods in a vector (optional)
60 MyAnalysisResult <- c(MyAnalysisResult1$p.value, MyAnalysisResult2$p.value)
61
62 #Evaluate the analysis results of Method_new and Method_old
63 MyResult1 <- MyEvaluation(MyAnalysisResult1)
64 MyResult2 <- MyEvaluation(MyAnalysisResult2)
65
66 #combine the results in a vector:
67 MyResult <- c(MyResult1, MyResult2)
68
69 # save stuff on export
70 setwd("/data1/claramuntj1/ALICEtutorial")
71 #Alternatively, you can also create your folder in the /data1/ALICEusername directory
72 #and set it as your working directory.
73
74 #IMPORTANT WARNING!!!
75 #When creating your own simulation study, make sure to create your own directory
76 #on the /data1/ALICEusername directory.
77 #For example: /data1/ALICEusername/projectName
78
79
80 # Write output (also possible to first save everything in a list object)
81
82 #optional to save the data
83 save(SimData, file =paste("Simdata","Row", RowOfDesign,
84     "Rep", Replication ,".Rdata" , sep =""))
85
86 #optional to save the full analysis result
87 save(MyAnalysisResult, file =paste("Analysis","Row", RowOfDesign,
88     "Rep", Replication ,".Rdata" , sep =""))
89
90 save(MyResult, file =paste("MyResult", "Row", RowOfDesign,
91     "Rep", Replication ,".Rdata" , sep =""))
92
93 #optional to save timing of analyses
94 save(time, file =paste("Time", "Row", RowOfDesign,
95     "Rep", Replication ,".Rdata" , sep =""))

```

6.2 The bash script used to run Jobs

Below you can find a bash script (let's call it *RunMySimulationSLURM.sh*) that is used on ALICE to run an R-script. The bash script 'starts' the R-script '*MainSimulationScriptSLURM.R*' with the command R CMD BATCH and includes some provided arguments: \$i and \$j. The \$ sign indicates that it is a replacement character, with a for loop you can change those values on the fly (see part 6.3). The values provided to these

replacement characters will be collected with the `commandArgs()` function in our R-script.

```
1
2 #!/bin/env bash
3 #SBATCH -J Sim_study
4 #SBATCH -N 1
5 #SBATCH --mem=512MB
6 #SBATCH --partition=cpu-short
7 #SBATCH --output=sim_study_%J.out
8 #SBATCH --error=sim_study_%J.err
9 #SBATCH --mail-user j.claramunt.gonzalez@fsw.leidenuniv.nl
10 #SBATCH --mail-type=BEGIN,END,FAIL
11
12 module load R/4.4.0-gfbb-2023a
13
14 Rscript MainSimulationScriptSLURM.R $i $j
```

For the small simulation example we are going to run, requesting 512MB of memory in 1 nodes should be enough.

For your own simulation study, we advise you to start by running only the row of your design which you expect to require more memory. If 512MB is enough there is no need to modify the script to run all the jobs. In case the memory is not enough (ALICE will return an error), increase it gradually, for example, with increments of 0.5Gb. Asking for too much memory will require a node with more memory, and consequently, your job will be more time in the queue. Also, make sure to select the right partition for your job. `cpu-short` (max 4 hours), `cpu-medium` (max 1 day) and `cpu-long` (max 7 days) are the partitions available for jobs that do not require a GPU. If you need a GPU, you can select the partition `gpu-short`, `gpu-medium` or `gpu-long`. Other partitions are available. Check <https://pubapps.lu.atlassian.net/wiki/spaces/HPCWIKI/pages/37519504/Partitions+on+ALICE>

6.3 Start your embarrassingly parallel jobs

We have an R-script with simulation code, *MainSimulationScriptSLURM.R*. We also have a bash script *RunMySimulationSLURM.sh* which ‘starts’ the R-script. But how do we actually start our simulation? We simply use the *sbatch* command in a double for-loop. Thus, our jobs are submitted to ALICE in a parallel fashion.

Before submitting multiple jobs, it is a good practice to submit only one first, in order to check that everything works properly. You can do it in the following way:

```
i=1 j=1 sbatch RunMySimulationSLURM.sh
```

In your project folder of the home directory you will observe two new files. One containing the R console output and one containing the possible errors. If the second one is empty, there were no errors. The error file can be used to spot the errors. Solve them before moving to the next step.

You can also check your saved output (different from the R console output) on the project folder you have created on the `exports/fsw` directory.

Now, it is time to submit multiple jobs. Let’s suppose that we want to submit all the replications for the first row of our design matrix. The code should be as follow:

```
for replication in {1..10}; do
i=1 j=$replication sbatch RunMySimulationSLURM.sh; done
```

Remember, for the current example the simulation design consists of two factors: Sample size (Samp) and effect size (Es). The factors have 4 and 3 levels respectively. So we have a 4x3 factorial design. And we want

to replicate this design $k = 10$ times. Thus, we will generate, analyze and evaluate a total of $4 \times 3 \times 10 = 120$ datasets. This can be done as follow:

```
for replication in {1..10}; do for rownumber in {1..12}; do
i=$rownumber j=$replication sbatch RunMySimulationSLURM.sh; done; done
```

Note 1: Remove the email arguments (lines 7 and 8) from *RunMySimulationSLURM.sh*. Otherwise you will receive 240 emails.

Note 2: In order to use *R* on ALICE, you first need to ‘load’ it. This can simply be done on the command line by typing `module load R/4.4.0-gfbb-2023a` (or select another *R* version available on ALICE)

You can submit the double for loop on the command line in ALICE. This double for loop will start $10 \times 12 = 120$ jobs that will run in parallel. Each job has no dependency on each other, meaning that you can run each row of the design completely separate. This is known as embarrassingly parallel computing and it saves you a lot of computation time!

6.3.1 Job Arrays (alternative to the for-loop)

A downside of using for-loops to submit jobs is that each job is submitted to the cluster individually. This is taxing for the head node, especially if the number of jobs is large. An alternative to the use of the for-loop for submitting jobs to ALICE is to use a job array.

A job array is a special job which consist of similar tasks that you want the cluster to execute. The primary advantage of using a job array is that the entire set of tasks is submitted to the queue as a single entity. The array as a whole is assigned a job identifier (column ‘JOBID’ in the output of `squeue`), and each task within the array is assigned an individual task identifier (column ‘JOBID’, after the job identifier). In the `squeue` output, waiting tasks are collapsed into a single row.

For example, if we have 1000 tasks queued this will appear as a single job with `JOBID_[1-1000]` (i.e. “tasks 1 through 1000 are waiting”), rather than as 1000 individual jobs. Running tasks are still displayed individually, so you can see on what node they are being executed, etc. The presence of both a job and task identifier makes it easy to request information on, or make changes to, both the individual tasks and the array as a whole.

Creating a job array is very simple: just include the ‘-array’ flag in your bash script, for example, the script ‘*RunMySimulationSLURM.sh*’ generating ‘*RunMySimulationJobArraySLURM.sh*’. An example on how to create a job array is as follows:

```
#!/bin/env bash
#SBATCH -J my_array
#SBATCH -N 1
#SBATCH --mem=512MB
#SBATCH --partition=cpu-short
#SBATCH --output=sim_study_%a.out
#SBATCH --error=sim_study_%a.err
#SBATCH --array=1-10

module load R/4.4.0-gfbb-2023a

i=1

Rscript MainSimulationScriptSLURM.R $i $SLURM_ARRAY_TASK_ID
```

%a in the output and error files is substituted by the `SLURM_ARRAY_TASK_ID` value. %A can be used if you prefer to include the `SLURM_ARRAY_JOB_ID` value.

Submitting this shell script to the cluster will create a job array, which consists of 10 tasks (`-array=1-10`).

Each task executes the file ‘MainSimulationScriptSLURM.R’. In this small example we are running the 10 repetitions of the first combination of parameters in the design.

It can be useful to save not only the results of a script, but also what is printed in the R console during execution. When using the Rscript command, you can easily do this by specifying a .Rout file. For example:

```
#!/bin/env bash
#SBATCH -J my_array
#SBATCH -N 1
#SBATCH --mem=512MB
#SBATCH --output=sim_study_%a.out
#SBATCH --error=sim_study_%a.err
#SBATCH --array=1-40

module load R/4.4.0-gfbb-2023a

Rscript my_script.R $SLURM_ARRAY_TASK_ID
```

This job array will save the R console output of each task to a .Rout file, which is a simple text file that can be read using e.g. cat.

In general, when running a job array, we do not want each task to do exactly the same thing: we want to vary experimental parameters, etc. Fortunately, the identifier for each task is stored in an environment variable called *SLURM_ARRAY_TASK_ID* (as we can observe in the previous example). We can obtain the value of this variable in R by using e.g. `as.numeric(Sys.getenv("SLURM_ARRAY_TASK_ID"))`, as in the following example:

```
1  # Experimental factors
2  f1 <- c(10, 20)
3  f2 <- c(0.2, 0.5)
4
5  # Replications
6  rep <- 1:10
7
8  # Matrix containing parameters for all individual tasks to perform
9  Design <- expand.grid(rep, f1, f2)
10
11 # Generate seeds
12 set.seed(123)
13 seed.list <- sample(.Machine$integer.max/2, size = nrow(Design))
14
15 # Obtain the Task ID
16 TID <- as.numeric(Sys.getenv("SLURM_ARRAY_TASK_ID"))
17
18 # Apply some function to obtain some result
19 set.seed(seed.list[TID])
20 result <- my_function(Design[TID,])
21
22 # Save the result
23 save(result, file=paste0("<path_to_folder>/my_result_", TID, ".RData"))
```

Submitting a job array as the one in the example will now execute this function 10 times, each with a unique combination of f1, f2 and seed.

7 Collecting the results

After all submitted jobs are done, you should collect all results from your output folder. Below you can find an R script (*ResultsCollecting.R*) that collects the right output and automatically stores the results in a matrix. This matrix can be used for further analyses such as an ANOVA.

Note that collecting the results can be done on ALICE. However, sometimes it is easier to first download the results to your own device and then collect the final results.

Note that this script uses the package gtools. You first need to install this package.

```
# Collect all results and put it in a matrix

# set directory with results
setwd("~/Desktop/ALICE/Data/")

library(gtools)

## original design

samp <- c(10, 20, 40, 80)
es <- c(0.2, 0.5, 0.8)
Design <- expand.grid(samp = samp, es = es)

# pattern matching with grep
files <- dir()
idx <- grep(pattern = "MyResult", files)
files <- files[idx]

# test to see mixedsort() does it's job
scram <- sample(1:120, 120, replace = F)
files <- files[scram]

files <- mixedsort(files)

###
RepIdxList <- list()
for(rep in 1:10){
  RepIdxList[[rep]] <- grep(pattern = paste("Rep",rep,sep = " "), files )
}

idx_to_remove <- RepIdxList[[1]] %in% RepIdxList[[10]]
RepIdxList[[1]] <- RepIdxList[[1]][!idx_to_remove]

idx <- unlist(RepIdxList)

files <- files[idx]

# rbind design k times and cbind files names
 #(so that you can check whether the right values are taken)

Results <- do.call(what = rbind, args = replicate(10, Design, simplify = F))
Results <- cbind(Results, files)

# the supply function loads the results and preserves the file name (in the rows).
# This is usefull for checking whether you did everything right
```

```

Res <- t( sapply(files, function(x) get(load(x) ) ) )

Results <- cbind(Results, Res)
colnames(Results) <- c("samp", "es", "files", "res1", "res2")

save(Results, file = "AllResults.Rdata")

```

8 Email notifications

When submitting a job you can use the `-mail-user` and `-mail-type` flags to send yourself email notifications. The `-mail-user` flag is used to specify your email address, while the `-mail-type` flag is used to specify when you want to be notified: at the beginning of the job (BEGIN) and/or at the end (END), or when the job fails (FAIL).

You will notice that the job array examples above do not include email notifications: this is because the flags applies to each task individually. It is generally not recommended to use begin- and end-notifications with large job arrays, otherwise you will get an email at the start and end of each individual task, which is not pleasant for both the mail server and your inbox (the same applies for the for-loop).

9 Installing R packages on ALICE

Option 1:

Start an individual R session on ALICE:

```

$ salloc -N1 -n1
$ module load R/4.4.0-gfbb-2023a (or another version)
$ R

```

Once R is open install the package you want as usual in R, using: `install.packages("")`.

Choose a CRAN mirror(if you have not included it as an argument of the `install.packages` function), and say that you want to create a personal folder to install the package when ALICE asks you (this will happen the first time you install a package).

Note: Once you have finished, remember to exit R using `q()`, and then, logout from the ALICE node typing 'exit'. Otherwise, the job will continue in the queue.

Option 2 (only works when you already have a personal folder):

The first step is to create a personal folder (if you do not have one yet). To do so, type on the command line:

```
mkdir /home/YOUR_DIRECTORY/R/x86_64-pc-linux-gnu-library/4.4 (or the corresponding R version)
```

Then, create the R script to install the packages, 'InstallPackages.R':

```
install.packages(c("ica", "Matrix"), Sys.getenv("R_LIBS_USER"), repos = "http://cran.case.edu")
```

Finally, generate the bash script 'InstallPackages.sh' to run the R script in ALICE.

```

#!/bin/env bash
#SBATCH -J installpackages
#SBATCH -N 1
#SBATCH --mem=512MB
#SBATCH --output=sim_study.out
#SBATCH --error=sim_study.err

module load R/4.4.0-gfbb-2023a

```

```
Rscript InstallPackages.R
```

How to run this script on ALICE: On the commandline type *sbatch InstallPackages.sh*.

10 Warning for writing shell scripts on Windows: line endings

If you are a Microsoft Windows user and are writing shell scripts (.sh) for execution on ALICE, make sure that your files have the correct Unix-style line endings, rather than the default Windows-style (see figure 4), otherwise your scripts may not execute properly.

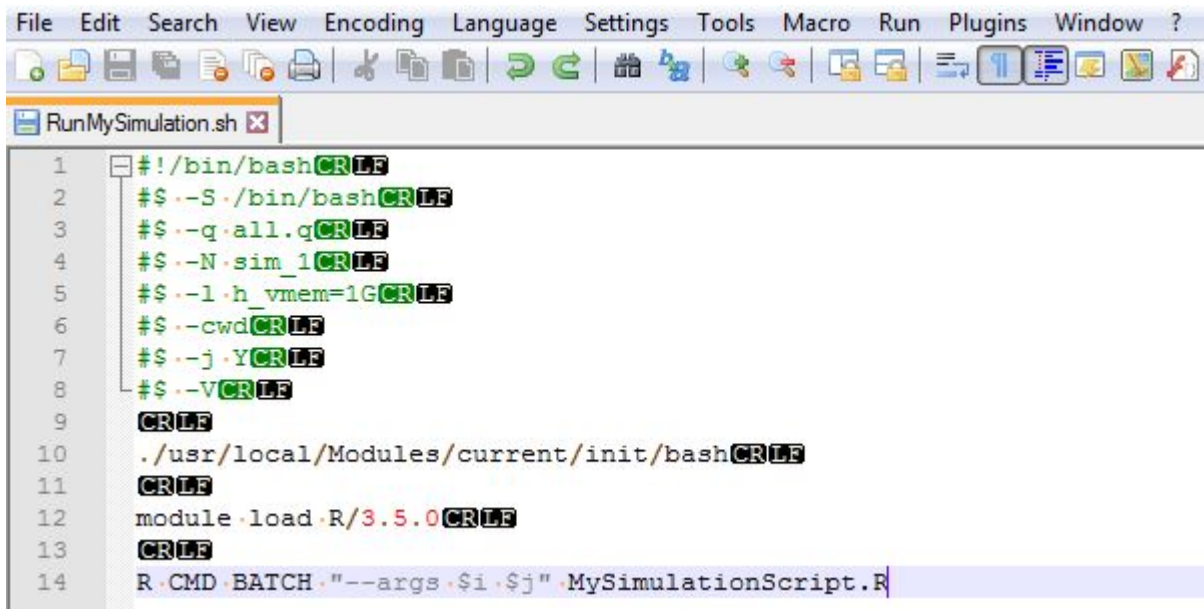


Figure 3: Windows line endings

Most decent text editors (e.g. Notepad++, Sublime Text) have an option to set the line endings or convert to Unix. In the figure 5 it is shown the way to correct it in Notepad++.

11 Useful Linux commands

pwd: It returns your working directory.

ls: It displays the content of your working directory.

ln -s: Create a shortcut to a folder.

cd: It changes your working directory to a new one given as parameter.

man: It opens the manual regarding the command given as parameter.

mkdir: It creates a new directory.

rmdir: It removes an empty directory. Use `rm -r` to delete the directory with the files in it.

nano: It opens nano, a text editor.

cp: It copies a directory. Add `-r` to copy the files in the folder.

exit: It exits from the terminal or ends the connection to ALICE.

#Matlab

In this current SLURM distribution of ALICE, both, Matlab and Octave (freeware alternative to Matlab), are available.

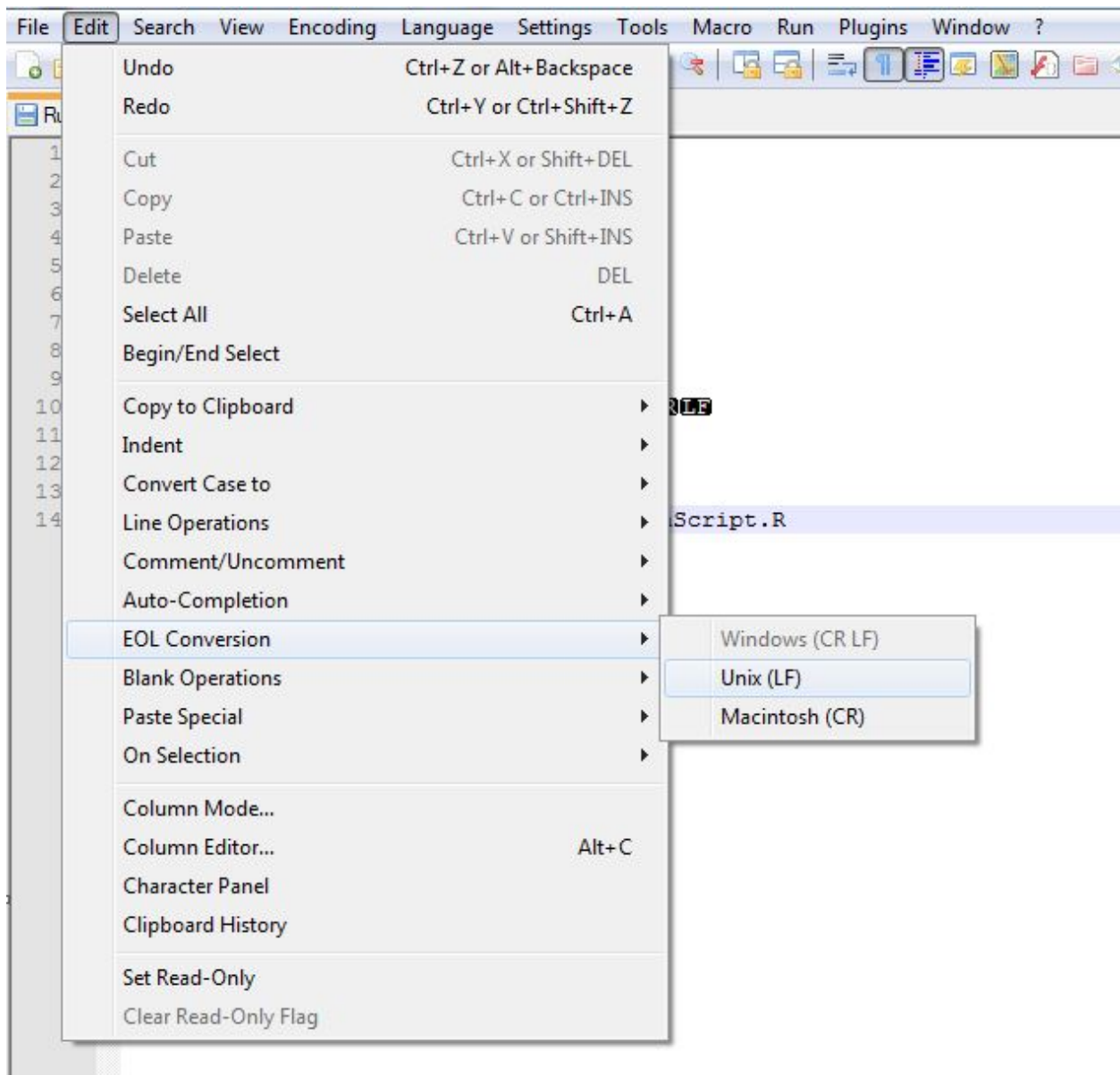


Figure 4: Modify Windows line endings with Notepad++

In order to perform the simulation study on Octave/Matlab, we need first to write analogous functions to *MyDataGeneration.R*, *Method_old.R*, *Method_new.R*, and *MyEvaluation.R*, as well as a script analogous to *MainSimulationScript.R* (let's call it *MainSimulationScript.m*).

Next, we need to slightly modify the .sh file. An example of it is the following:

```
#!/bin/bash
#SBATCH -J matlab_example
#SBATCH -N 1
#SBATCH --mem=512MB
#SBATCH --partition=cpu-short
#SBATCH --output=matlab_example.out
#SBATCH --error=matlab_example.err

module load MATLAB/2020b

matlab MainSimulationScript.m
```

For running in parallel or using job arrays to run your script go to chapter 6.

Note: To run octave, substitute the last two lines of the previous .sh file. First load octave instead of matlab. Second, substitute the command matlab by the command octave.